Here's the thing though: PHP has patterns from both functional and object-oriented paradigms. It has become increasingly more FP-like over time, even as it added more OOP features. I am advocating for neither over the other, but being aware of the features of these paradigms will, I believe, help you to get the best out of PHP.

Let's review some of the features of functional programming, given that we've just spent a fairly hefty chapter dissecting how functions work!

First-Class and Higher-Order Functions

Functions in a programming language are said to be "first-class citizens" if they can be treated like any other data. A language supports first-class functions if it allows them to be used as arguments to other functions, returned by other functions, or stored in variables. As we have seen earlier in the chapter, we can store anonymous functions in PHP with a variable, we can send functions to other functions as callbacks (hence the `callable` data type), and we can obtain one function from another. A "higher-order" function is one that takes one or more functions as an argument and/or returns a function.

This is a powerful feature, because it allows functions to be composed from other functions (and in due course we'll see how objects can be composed from other objects, an equally powerful tool). Examples of higher-order functions in PHP are `array_map()` and `array_reduce()`.

Pure Functions

A function is "pure" if it meets the following criteria: it has no side effects (no changes to local static variables, global variables, reference arguments, or I/O streams – so that's PHP buggered then), and it always returns the same output for the same input. In a truly FP language, no data is ever mutated (instead, new data is created, which is why FP languages are sometimes less memory efficient). Trying to adopt a pure functional approach for your code is going to pose serious challenges for a language invented to drive websites, and I don't suggest you try.

However, one great benefit of pure functions is that they are incredibly easy to test. The fewer side effects a function has, the easier it is to write tests for. Keeping side effects to a minimum *will* make your life easier, I promise.